Faculty of Sciences

Department of Computer Sciences

Master thesis

Procedural Generation of Solvable Levels in Multi-Agent Reinforcement Learning Environment

Author:
Charels Hugo
hugo.charels@ulb.be

Supervisors:
Lenaerts Tom
Molinghen Yannick

Academic year 2025-2026

Contents

1 Introduction ... 1
1.1 Context ... 1
1.2 Motivation ... 1
1.3 Problem Statement ... 2
1.4 Contributions ... 3
1.5 Thesis Structure ... 3
2 Background ... 5
2.1 Multi-Agent Reinforcement Learning ... 5
2.1.1 Single-Agent Reinforcement Learning ... 5
2.1.2 Extension to Multiple Agents ... 5
2.1.3 Challenges in Cooperative MARL ... 6
2.2 The Laser Learning Environment ... 6
2.2.1 Structure and Mechanics ... 6
2.2.2 An Illustrative Example ... 7
2.2.3 Relevance as a Benchmark ... 8
2.3 Procedural Content Generation ... 8
2.4 Boolean Satisfiability ... 9
2.4.1 The SAT Problem ... 9
2.4.2 Modern SAT Solvers ... 9
2.4.3 Complexity of LLE Solvability ... 9
3 Related Work ... 11
4 Methods ... 12
4.1 Problem Formalization ... 12
4.1.1 LLE Level ... 12
4.1.2 Joint Trajectory ... 12
4.1.3 Solvability ... 12
4.1.4 Cooperation Requirement ... 13
4.2 Solver by Reduction to SAT ... 13
4.2.1 Definition of Sets ... 13
4.2.2 Definition of Variables ... 14
4.2.3 Constraints and Logical Encoding ... 14
4.3 Cooperation Detection ... 17
4.3.1 Strict Laser Semantics ... 17
4.3.2 Why This Captures Cooperation ... 17
4.3.3 Formal Theorem and Proof ... 17
4.3.4 Practical Algorithm ... 18
4.4 Level Generators ... 18
4.4.1 Design Pattern ... 18
4.4.2 Random Solvable Generator ... 18
4.4.3 Constrained Random Solvable Generator ... 19
4.4.4 Random Cooperative Generator ... 19
4.4.5 Constrained Random Cooperative Generator ... 19
4.4.6 Summary ... 19
4.5 Benchmarking Setup ... 20
5 Experiments ... 21
5.1 Experimental Setup ... 21
5.1.1 Clause Count Analysis ... 21

5.1.2 Test Levels 21
5.2 Results 22
5.2.1 Clause Count 22
5.2.2 Solving Time 22
5.3 Discussion 23
6 Conclusion 24
Bibliography 25

Chapter 1

Introduction

1.1 Context

Reinforcement Learning (RL) has established itself as a powerful paradigm for training autonomous agents to make sequential decisions by interacting with an environment. In single-agent settings, an agent repeatedly observes the state of the world, selects an action, and receives a scalar reward signal that it tries to maximise over time. This framework has produced remarkable results in domains ranging from board games to robotic control.

Multi-Agent Reinforcement Learning (MARL) extends this paradigm to settings in which several agents act simultaneously within a shared environment. The agents may compete, cooperate, or coexist independently, and their policies influence one another's learning dynamics. Cooperative MARL, in particular, focuses on scenarios where a group of agents must jointly achieve a shared goal — a setting that arises naturally in applications such as robot swarm coordination, multi-robot task allocation, and cooperative strategy games.

Cooperative tasks introduce challenges that go beyond single-agent RL. Agents must not only explore a potentially vast joint action space, but they must do so while accounting for the behaviour of their teammates. Reward signals are often sparse: in many cooperative settings, the team receives a reward only upon completing the full task, leaving agents with no intermediate feedback to guide exploration. This makes the discovery of coordinated strategies especially difficult when the required joint behaviour is long and precisely ordered.

A dimension of cooperative MARL that is frequently underestimated is the role of the training environment itself. The structure of the levels or scenarios in which agents are trained directly determines what coordination behaviours they can discover and practise. A poorly designed environment — one that is unsolvable, trivially solvable by a single agent, or repetitive in structure — yields limited training signal and constrains generalisation. Conversely, a well-designed set of environments can expose agents to a rich variety of coordination challenges, accelerate learning, and produce more robust policies. Designing such environments at scale, however, is a labour-intensive process when done manually.

1.2 Motivation

Procedural Content Generation (PCG) offers an alternative: the algorithmic, automatic creation of environments according to user-defined criteria. Originally developed for video game level design, PCG has since been applied to simulation-based agent training, where it enables the production of large, diverse sets of environments without manual intervention. In the MARL context, PCG can supply a steady stream of novel levels during training, reducing the risk of overfitting to a fixed set of hand-crafted scenarios and supporting generalisation across varied task structures.

The central difficulty in applying PCG to cooperative MARL is ensuring that generated environments are meaningful: not only structurally well-formed, but genuinely solvable, and genuinely cooperative. These are non-trivial requirements.

Solvability in a multi-agent setting is more demanding than in single-agent settings. It is not sufficient for each agent to have a path to its goal; the agents must be able to reach their goals simultaneously, following a joint sequence of actions that is consistent with all environmental constraints — including dynamic obstacles such as laser beams whose activity depends on the current positions of other agents. Testing whether such a joint solution exists is computationally non-trivial, and naively generating random environments yields mostly unsolvable configurations, especially as the grid size and number of agents grow.

Cooperation must be structurally enforced. A level that can be solved by agents acting independently — without any agent ever assisting another — provides no training signal for cooperative behaviour. For a level to serve as a useful cooperative training instance, it must have the property that no agent can reach its goal unless at least one other agent performs a supporting action. Simply generating solvable levels does not guarantee this: the generator must be designed or constrained to produce configurations in which cooperation is a structural necessity, not an option.

This thesis addresses both requirements together. We focus on the Laser Learning Environment (LLE), a 2D grid-based cooperative benchmark for MARL, as our primary instantiation. LLE is particularly well-suited to this study: its laser-blocking mechanics create precisely the kind of inter-agent dependencies that make cooperation structurally necessary in certain configurations, and its well-defined grid representation is amenable to formal analysis.

1.3 Problem Statement

The central problem this thesis addresses is the following: how can we automatically generate LLE levels that are provably solvable and provably require inter-agent cooperation?

We formalise three properties that a generated level should satisfy:

1. **Solvability** — the level admits at least one valid joint action sequence through which all agents reach their respective exits. This is a necessary baseline: a level that cannot be completed is useless for training.
2. **Cooperation requirement** — the level cannot be solved without at least one agent performing a cooperative act (specifically, blocking a laser of its own colour to allow a teammate to pass). This property ensures that the level is not trivially solvable by independent agent behaviour.
3. **Learnability** — the solution is discoverable by MARL algorithms through exploration. While solvability and cooperation can be verified formally, learnability depends on the agent architecture, the reward structure, and the exploration strategy, and remains an open problem

outside the scope of this thesis. We treat it as a desirable property and discuss it in terms of structural affordances that facilitate discovery.

The approach we adopt to address the first two properties is a reduction to Boolean Satisfiability (SAT): we encode the constraints of the LLE level as a propositional formula in conjunctive normal form (CNF), and delegate the solvability check to a modern SAT solver. The cooperation property is detected via a second SAT call on a stricter variant of the encoding. Both calls are embedded in a generation loop that produces levels until the desired properties are satisfied.

1.4 Contributions

This thesis makes the following contributions:

- **A SAT-based solver for LLE solvability.** We provide a complete and correct CNF encoding of the LLE problem over a bounded time horizon T. Given a level and a horizon, the solver returns SAT (with a satisfying assignment that encodes a valid joint trajectory) or UNSAT (certifying that no solution exists within T steps). The encoding is described in full in .
- **A formal cooperation detector.** We define a strict variant of the LLE semantics in which agents are no longer immune to lasers of their own colour. We prove that a level requires cooperation if and only if the standard encoding is satisfiable and the strict encoding is unsatisfiable. This gives a decision procedure for the cooperation property based on two SAT calls ().
- **A family of procedural level generators.** Building on the solver and cooperation detector, we design four generators of increasing sophistication: a random solvable generator, a constrained random solvable generator, a random cooperative generator, and a constrained random cooperative generator. All generators offer formal guarantees on the properties of their output ().
- **An empirical evaluation.** We benchmark the generators and solver across a range of grid sizes and agent counts, measuring solve time, acceptance rate, cooperation rate, and level diversity (,).

The approach developed here is specific to LLE in its SAT encoding — the laser mechanics, color-matching rules, and exit conditions are all embedded in the formula structure. However, the broader methodology — reducing solvability and cooperation to formal properties, verifying them via a constraint solver, and embedding the verifier in a generation loop — is general and can be adapted to other grid-based cooperative MARL environments, provided an appropriate encoding is designed for the target environment.

1.5 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 — Background introduces the technical foundations needed to follow the thesis: the MARL framework and its cooperative variant, a detailed description of the Laser Learning Environment, an overview of Procedural Content Generation and its main paradigms, and an introduction to Boolean Satisfiability including its complexity-theoretic role in the context of LLE.

Chapter 3 — Related Work surveys existing work on PCG for game environments, solvability-aware generation, and procedural generation for MARL training.

Chapter 4 — Methods is the core of the thesis. It formalises the LLE level structure, solvability, and cooperation requirement; presents the full SAT encoding and its correctness argument; introduces

the cooperation detection theorem and its proof; describes the four generators; and defines the benchmarking protocol.

Chapter 5 — Experiments presents the empirical results: solver performance, generator acceptance rates, cooperation rates, and level diversity across a range of configurations.

Chapter 6 — Conclusion summarises the contributions, discusses limitations of the current approach, and outlines directions for future work, including adapting the methodology to other cooperative MARL environments and formally addressing the learnability property.

Chapter 2

Background

2.1 Multi-Agent Reinforcement Learning

2.1.1 Single-Agent Reinforcement Learning

Reinforcement Learning (RL) is a framework for sequential decision-making in which an agent interacts with an environment over a series of discrete time steps. At each step t, the agent observes a state s_t from the state space $\mathcal{S}$, selects an action a_t from its action space $\mathcal{A}$, and receives a scalar reward $r_t \in \mathbb{R}$ from the environment. The environment then transitions to a new state s_{t+1} according to a transition function $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$, where $\Delta(\mathcal{S})$ denotes the set of probability distributions over $\mathcal{S}$. This structure defines a Markov Decision Process (MDP), characterised by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function and $\gamma \in [0, 1)$ is a discount factor controlling the relative weight of future rewards.

The agent's goal is to learn a policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ that maximises the expected discounted cumulative reward, or return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

RL algorithms differ in how they estimate and optimise this objective — through value function approximation (Q-learning, DQN), policy gradient methods (REINFORCE, PPO), or actor-critic architectures — but all share the same underlying MDP structure.

2.1.2 Extension to Multiple Agents

Multi-Agent Reinforcement Learning (MARL) extends the single-agent framework to settings involving $n \geq 2$ agents acting simultaneously within a shared environment. The standard formalisation is the Markov Game (also called a Stochastic Game), defined by the tuple $\left(\mathcal{S}, \{\mathcal{A}^i\}_{i=1}^n, T, \{R^i\}_{i=1}^n, \gamma\right)$, where $\mathcal{A}^i$ is the action space of agent i and R^i is its individual reward function. At each time step, every agent selects an action independently, the joint action $\boldsymbol{a}_t = (a_t^1, ..., a_t^n)$ determines the state transition and each agent's reward.

Depending on the structure of the reward functions, Markov Games span a spectrum from fully competitive (zero-sum games, where agents' interests are directly opposed) to fully cooperative (team games, where all agents share the same reward). Intermediate settings — with a mix of individual and shared incentives — also arise in practice. This thesis focuses exclusively on the cooperative setting.

In cooperative MARL, all agents share a single reward signal: $R^1 = R^2 = ... = R^n = R$. The team's objective is to find a joint policy $\boldsymbol{\pi} = (\pi^1, ..., \pi^n)$ that maximises the shared return. Cooperation is thus not assumed but must emerge from learning: agents must discover that coordinating their actions leads to higher team rewards than acting independently.

2.1.3 Challenges in Cooperative MARL

Cooperative MARL introduces a set of challenges that are absent or less severe in single-agent settings.

Non-stationarity. From the perspective of any individual agent, the environment appears non-stationary: other agents are simultaneously updating their policies, causing the effective dynamics observed by each agent to change over time. This violates the stationarity assumption underlying most single-agent RL theory and can destabilise learning.

Exponential joint action space. The joint action space grows exponentially in the number of agents. Even when each agent has a small individual action space, the number of possible joint actions becomes intractable for large teams. This makes centralised control infeasible and motivates decentralised execution paradigms.

Credit assignment. When agents share a single reward signal, attributing each agent's contribution to the team outcome becomes difficult. Poor credit assignment leads to agents converging to suboptimal policies in which some agents free-ride on the efforts of others.

Sparse rewards and exploration. In many cooperative tasks, the team reward is issued only upon completing the full task. Agents must therefore explore long sequences of joint actions without intermediate feedback. In environments where the rewarding joint behaviour is rare or precisely ordered, pure random exploration is ineffective and training can stall entirely.

This last challenge is particularly acute in environments like LLE, where agents must execute carefully synchronised cooperative sequences with no intermediate reward — a setting known as the zero-incentive bottleneck problem, described in the following section.

2.2 The Laser Learning Environment

The Laser Learning Environment (LLE) is a 2D grid-based cooperative puzzle game designed as a benchmark for MARL algorithms. It supports between one and four agents and is built around a set of mechanics that make cooperative coordination both necessary and difficult to learn.

2.2.1 Structure and Mechanics

An LLE level is played on a rectangular grid of height H and width W. Each cell is either a floor tile, a wall, a laser source, or an exit. The key components are as follows.

Agents. Each agent is assigned a unique colour from the set $\mathcal{A}$. At the start of an episode, agent c is placed at its designated starting cell $s(c)$. At each time step, an agent may stay in place or move to an adjacent non-wall cell. Agents cannot occupy the same cell simultaneously.

Lasers. Each laser source $(c, d, p) \in \mathcal{S}$ emits a beam of colour c in direction $d \in \{N, S, E, W\}$ starting from cell p. The beam propagates cell by cell in direction d until it is blocked by a wall or, crucially, by an agent. The key mechanic governing laser interaction is **colour matching**: a beam of colour c is blocked only by an agent of colour c. Any agent of a different colour that enters a cell traversed by a beam of colour c is eliminated. This creates a strict interdependency: each laser can only be neutralised by one specific agent.

Exits. Each agent c has a designated exit cell $\mathcal{E}(c)$. The episode is completed successfully when every agent occupies its exit cell simultaneously. This synchronisation requirement prevents agents from solving the level independently: an agent that reaches its exit early must remain there, possibly blocking other agents, until all teammates arrive.

Zero-incentive bottlenecks. A distinctive property of LLE is that intermediate cooperative acts — such as positioning to block a laser for a teammate — carry no immediate reward. The reward is issued only when all agents complete the level. An agent that sacrifices its own optimal path to assist a teammate receives no direct signal that this behaviour is beneficial. This creates exploration bottlenecks: agents have no local incentive to adopt cooperative strategies, and discovering the required joint behaviour through random exploration is exceedingly unlikely, especially in larger levels.

2.2.2 An Illustrative Example

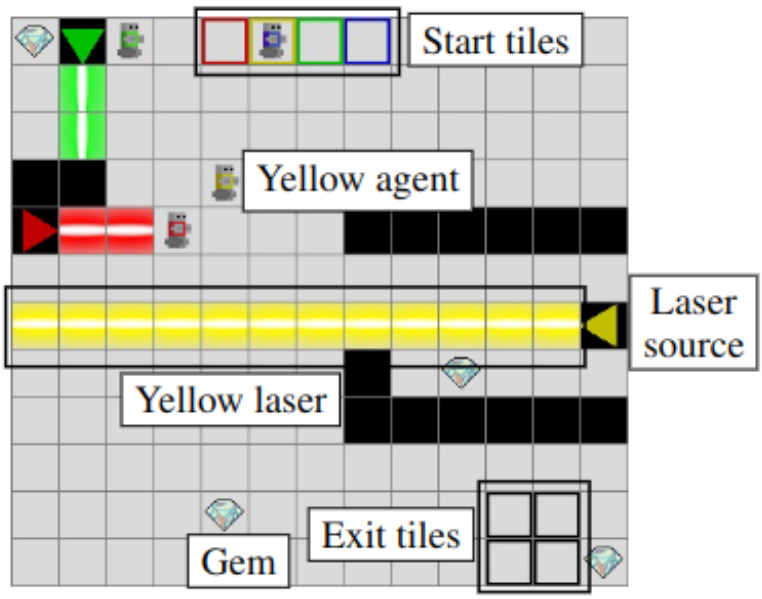

Figure 1: An annotated LLE level. Each agent must reach the exit of its own colour. Laser beams (coloured lines) block movement for agents of a different colour. Agent 0 (blue) must block the blue laser to allow agent 1 (orange) to pass — a cooperative act that yields no intermediate reward.

The figure above illustrates a typical LLE level. The blue agent must position itself in the path of the blue laser, blocking it so that the orange agent can pass through cells the beam would otherwise reach. This blocking act is purely beneficial to the orange agent: the blue agent is immune to its own

laser and gains nothing directly from the manoeuvre. The level cannot be completed without this cooperative step, yet no reward is given for performing it.

2.2.3 Relevance as a Benchmark

LLE has been used to evaluate several state-of-the-art value-based MARL algorithms, including methods augmented with prioritised experience replay, n-step returns, and intrinsic curiosity via random network distillation . Despite these enhancements, agents frequently fail to escape coordination bottlenecks in LLE, even when they achieve near-perfect coordination in more conventional cooperative benchmarks.

This combination of properties — formal solvability structure, strict coordination requirements, sparse rewards, and zero-incentive bottlenecks — makes LLE an ideal environment for studying the relationship between level design and agent learning. It is the primary instantiation of our framework, though the methodology developed here is designed with generalisability to other grid-based cooperative environments in mind.

2.3 Procedural Content Generation

Procedural Content Generation (PCG) refers to the algorithmic creation of content — levels, maps, textures, narratives, or entire game worlds — with limited or no direct human input. PCG has a long history in game development, where it serves to increase replayability, reduce production costs, and produce content at scales that manual design cannot achieve. More recently, PCG has been adopted in simulation-based agent training as a way to supply diverse and varied environments during learning.

PCG methods can be broadly organised into three paradigms.

Constructive methods generate content in a single forward pass, without backtracking or verification. Examples include noise-based generators (Perlin noise, cellular automata) that produce cave-like or organic layouts, grammar-based systems that expand structural rules into level geometry, and Wave Function Collapse, which tiles levels from locally consistent patterns. Constructive methods are fast and scalable, but offer no formal guarantees: a generated level may be unsolvable, disconnected, or trivially easy.

Search-based methods frame content generation as an optimisation problem and explore the space of possible levels using heuristic search or evolutionary algorithms. A fitness function evaluates candidate levels against desired properties — solvability, playability, difficulty — and the search is guided toward high-fitness regions. Search-based methods can incorporate complex objectives but are computationally expensive and depend heavily on the quality of the fitness function.

Constraint-based methods encode the desired properties of the content as a constraint satisfaction problem (CSP) or a propositional formula, and use a solver to find a level that satisfies all constraints. This approach offers formal guarantees: if a solution is found, it is certified to satisfy every specified property. The cost is the computational overhead of solving the constraint system, which grows with the complexity of the level.

This thesis adopts the constraint-based paradigm, using Boolean Satisfiability as the underlying solver. The choice is motivated by the need for formal solvability and cooperation guarantees — properties that constructive and search-based methods cannot provide directly.

2.4 Boolean Satisfiability

2.4.1 The SAT Problem

The Boolean Satisfiability problem (SAT) is the canonical NP-complete decision problem. Given a propositional formula φ over a set of Boolean variables $x_1, ..., x_n$, the question is: does there exist a truth assignment $\nu : \{x_1, ..., x_n\} \to \{\text{true}, \text{false}\}$ such that $\varphi(\nu) = \text{true}$?

SAT solvers operate on formulas in **Conjunctive Normal Form** (CNF): a conjunction of clauses, where each clause is a disjunction of literals (a variable or its negation):

$$\varphi = \bigwedge_{i=1}^{m} \bigvee_{j} \ell_{i,j}$$

Any propositional formula can be converted to an equisatisfiable CNF formula in polynomial time using the Tseitin transformation. When a formula is satisfiable, a solver returns a satisfying assignment (a **model**); when it is not, it returns UNSAT with a proof of unsatisfiability.

2.4.2 Modern SAT Solvers

Early SAT solvers were based on the Davis-Putnam-Logemann-Loveland (DPLL) algorithm, a backtracking search procedure augmented with unit propagation and pure literal elimination. Modern solvers implement Conflict-Driven Clause Learning (CDCL), which extends DPLL with two key innovations: **clause learning** (recording the reason for each conflict as a new clause, preventing the solver from revisiting the same conflict) and **non-chronological backtracking** (jumping back further in the search tree based on the learned clause). CDCL solvers also employ sophisticated heuristics for variable ordering and restart policies that dramatically reduce the search space in practice.

Despite SAT being NP-complete in the worst case, modern CDCL solvers routinely handle industrial instances with millions of variables and clauses within seconds. This empirical tractability is the primary motivation for using SAT as the backend for our solvability checker.

2.4.3 Complexity of LLE Solvability

The SAT problem is NP-complete by the Cook-Levin theorem : every problem in NP can be reduced to SAT in polynomial time, and SAT is itself in NP. This places SAT at the boundary of what is efficiently verifiable but not (provably) efficiently solvable in the worst case.

The solvability problem for LLE — given a level L and a time horizon T, does a valid joint trajectory of length at most T exist? — is in **NP**: a candidate trajectory can be verified in polynomial time by simulating the joint execution and checking that all agents reach their exits without violating any constraint.

The reduction we present in encodes this problem as a CNF formula in polynomial time with respect to the level parameters (H, W, $|\mathcal{A}|$, T). This establishes that LLE solvability is polynomial-time many-one reducible to SAT:

$$\text{LLE-Solvability} \leq_p \text{SAT}$$

This means LLE solvability is **at most as hard as SAT**: any algorithm for SAT immediately yields an algorithm for LLE solvability with only polynomial overhead.

Whether LLE solvability is **NP-hard** — i.e., whether SAT is also reducible to LLE — remains an **open question**. Establishing NP-hardness would require constructing a polynomial-time reduction from an NP-hard problem (such as SAT or 3-SAT) to LLE solvability, showing that LLE solvability encodes arbitrary constraint satisfaction. Such a result would place LLE solvability in the class of NP-complete problems, meaning it is among the hardest problems in NP and no polynomial-time algorithm exists unless P = NP.

In any case, the practical consequence of the reduction is clear: our SAT-based solver inherits the strong empirical performance of modern CDCL solvers, making it effective on the LLE instances encountered in practice, even if the worst-case complexity is high.

Chapter 3

Related Work

Chapter 4

Methods

4.1 Problem Formalization

We now formally define the objects and properties at the center of this thesis. This section operates at the level of the game semantics; the SAT encoding of these properties follows in .

4.1.1 LLE Level

An LLE level is a tuple $L = (H, W, \mathcal{A}, s, \mathcal{W}, \mathcal{S}, \mathcal{E})$ where:

- $H, W \in \mathbb{N}^+$: height and width of the grid.
- $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$: set of all grid positions.
- $\mathcal{A} = \{0, 1, ..., n_a - 1\}$: set of agents, each identified by a color. $n_a \geq 1$.
- $s : \mathcal{A} \to P$: initial position of each agent.
- $\mathcal{W} \subseteq P$: set of wall positions.
- $\mathcal{S} \subseteq \mathcal{A} \times D \times P$: set of laser sources, where $D = \{N, S, E, W\}$. A source $(c, d, p) \in \mathcal{S}$ emits a laser of color c in direction d from position p.
- $\mathcal{E} : \mathcal{A} \to P$: exit position for each agent. We require $|\{\mathcal{E}(c) \mid c \in \mathcal{A}\}| = n_a$ (distinct exits).

4.1.2 Joint Trajectory

A **joint trajectory** of length T is a sequence of joint positions $\sigma = (p_0, p_1, ..., p_T)$ where each $p_t : \mathcal{A} \to P$ assigns a position to every agent at time step t.

A joint trajectory is **valid** if it satisfies:

1. **Initialization:** $p_0(c) = s(c)$ for all $c \in \mathcal{A}$.
2. **Movement:** For all $t < T$ and all $c \in \mathcal{A}$, $p_{t+1}(c)$ is reachable from $p_{t(c)}$ in one step (adjacent or same cell, not a wall, not a laser source cell).
3. **No collision:** $p_{t(c_1)} \neq p_{t(c_2)}$ for all t and all $c_1 \neq c_2$.
4. **Laser safety:** No agent c_1 occupies a cell at time t where a laser of color $c_2 \neq c_1$ is active (laser activity is determined by agent positions via the propagation rules).
5. **Stay on exit:** Once an agent reaches its exit, it remains there.

4.1.3 Solvability

A level L is **solvable** with horizon T if there exists a valid joint trajectory σ of length T such that $p_{T(c)} = \mathcal{E}(c)$ for all $c \in \mathcal{A}$.

A level is **solvable** (without qualification) if it is solvable for some finite T.

Lorem ipsum dolor.

4.1.4 Cooperation Requirement

Informally, a level **requires cooperation** if no agent can reach its exit without the help of at least one other agent — specifically, without another agent blocking a laser on its behalf.

We make this precise via the **strict laser semantics**: in the strict variant, agents are no longer immune to lasers of their own color. Every agent is blocked by every laser regardless of color matching. A valid trajectory under strict semantics is a **strict trajectory**.

Lorem ipsum dolor.

Definition (Cooperation Requirement). A level L is said to **require cooperation** if:

1. L is solvable (under standard semantics), and
2. L admits no valid strict trajectory reaching all exits.

The formal proof that this definition correctly captures the intuitive notion of cooperation is given in .

(a) Unsolvable
no valid joint trajectory

(b) Solvable, no cooperation
agents reach exits independently

(c) Solvable and cooperative
laser blocking required

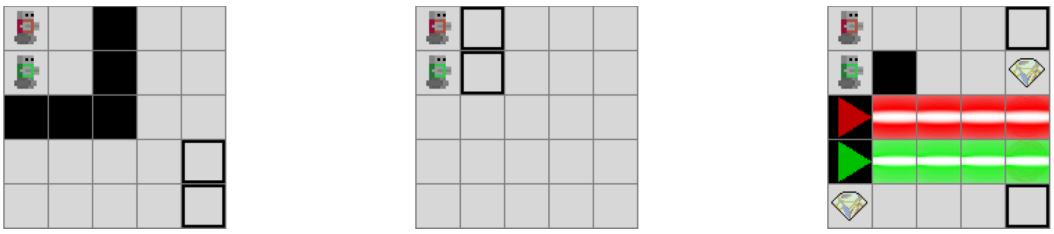

Figure 2: The three level categories defined by the solvability and cooperation properties.

4.2 Solver by Reduction to SAT

4.2.1 Definition of Sets

We consider a rectangular grid of height H and width W. Positions are identified by pairs (x, y), where $x \in \{0, ..., W-1\}$ denotes the column index and $y \in \{0, ..., H-1\}$ denotes the row index. The following sets are used throughout.

- $C = \{0, 1, ..., n_a - 1\}$: set of agent colors, where $n_a \geq 1$ is the number of agents.

- $D = \{N, S, E, W\}$: set of laser beam directions (North, South, East, West).
- $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$: set of all grid positions.
- $\mathcal{W} \subseteq P$: set of wall positions.
- $\mathcal{S} \subseteq C \times D \times P$: set of laser sources; $(c, d, p) \in \mathcal{S}$ denotes a laser of color c shooting in direction d from position $p \in P$.
- $\mathcal{E} \subseteq P$: set of exit positions, with $|\mathcal{E}| = n_a$.
- $T = \{0, 1, ..., T_{\max}\}$: set of discrete time steps.

We also write $\text{start}(c) \in P$ for the initial position of agent $c \in C$.

4.2.2 Definition of Variables

We introduce three families of propositional variables.

- $a_{c,x,y,t}$: true iff agent $c \in C$ occupies position $(x, y) \in P$ at time step $t \in T$.
- $b_{c,d,x,y,t}$: true iff the beam of laser source $(c, d, -) \in \mathcal{S}$ is active at position $(x, y) \in P$ at time $t \in T$.
- $l_{c,x,y,t}$: true iff a laser of color $c \in C$ is active at position $(x, y) \in P$ at time $t \in T$.

4.2.3 Constraints and Logical Encoding

We now formalize the constraints ensuring the correctness of the reduction. Each group of clauses corresponds to one logical component of the problem.

1. **Initialization**

   - **Agents:**
     Each agent $c \in C$ is placed at its designated starting position $\text{start}(c)$ at $t = 0$; all other positions are unoccupied by that agent:

$$\bigwedge_{c \in C} \bigwedge_{(x,y) \in P} \begin{cases} a_{c,x,y,0} & \text{if } (x, y) = \text{start}(c) \\ \neg a_{c,x,y,0} & \text{otherwise} \end{cases}$$

   - **Laser sources:**
     For each laser source $(c, d, (x_s, y_s)) \in \mathcal{S}$, the beam is always active at its origin, at every time step:

$$\bigwedge_{(c,d,(x_s,y_s)) \in \mathcal{S}} \bigwedge_{t \in T} b_{c,d,x_s,y_s,t}$$

2. **Agent Movements**

   We define the set of positions reachable from (x, y) in one step as (x, y) itself together with its four grid neighbors, excluding walls and laser source positions:

$$\begin{aligned} \text{next}(x, y) = \{(x', y') \in \{(x, y), (x, y-1), (x+1, y), (x, y+1), (x-1, y)\} \mid \\ (x', y') \in P, (x', y') \notin \mathcal{W}, (x', y') \notin \{p \mid (c, d, p) \in \mathcal{S}\}\} \end{aligned}$$

   - **Forward consistency:**
     If agent c is at position (x, y) at time t, it must be at some position in $\text{next}(x, y)$ at time $t + 1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t} \rightarrow \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t+1}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} \neg a_{c,x,y,t} \vee \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t+1}$$

Two methods are proposed to enforce that each agent occupies at most one position at each time step.

- **Global uniqueness:**
  No two distinct positions can both be occupied by agent c at time t:

$$\bigwedge_{c\in C}\bigwedge_{t\in T}\bigwedge_{(x_1,y_1)\in P}\bigwedge_{\substack{(x_2,y_2)\in P,\\ (x_2,y_2)\neq(x_1,y_1)}} \neg a_{c,x_1,y_1,t} \vee \neg a_{c,x_2,y_2,t}$$

- **Local uniqueness:**
  No two distinct positions in $\text{next}(x, y)$ can simultaneously be occupied by agent c at time $t + 1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\bigwedge_{(x',y')\in \text{ next}(x,y)}\bigwedge_{\substack{(x'',y'')\in \text{ next}(x,y),\\ (x'',y'')\neq(x',y')}} \neg a_{c,x',y',t+1} \vee \neg a_{c,x'',y'',t+1}$$

  Backward consistency:
  If agent c is at position (x, y) at time $t + 1$, it must have been at some position in $\text{next}(x, y)$ at time t:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t+1} \rightarrow \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} \neg a_{c,x,y,t+1} \vee \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t}$$

- **No simultaneous occupation:**
  Two distinct agents cannot share the same position at the same time, nor can they move to a position already occupied by the other agent:

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t\in T} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t+1} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t+1}$$

- **Victory condition:**
  Each exit must be occupied by at least one agent at time $T_{\max}$. Since $|\mathcal{E}| = n_a$ and agents cannot share positions, this enforces a bijection between agents and exits:

$$\bigwedge_{(x,y)\in\mathcal{E}} \bigvee_{c\in C} a_{c,x,y,T_{\max}}$$

- **Stay on exit:**
  Once an agent reaches an exit, it remains there for all subsequent time steps:

$$\bigwedge_{c\in C} \bigwedge_{(x,y)\in\mathcal{E}} \bigwedge_{t=0}^{T_{\max}-1} \neg a_{c,x,y,t} \vee a_{c,x,y,t+1}$$

3. **Laser Activity**

We define $\text{next}_{d(x,y)}$ as the position immediately adjacent to (x, y) in direction d:

$$\text{next}_d(x,y) = \begin{cases} (x, y-1) \text{ if } d = N \\ (x+1, y) \text{ if } d = E \\ (x, y+1) \text{ if } d = S \\ (x-1, y) \text{ if } d = W \end{cases}$$

- **Walls block beams:**
  A beam cannot be active at a wall position:

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{(x,y)\in\mathcal{W}} \bigwedge_{t\in T} \neg b_{c,d,x,y,t}$$

- **Beam propagation:**
  For each laser source $(c, d, -) \in \mathcal{S}$ and each non-wall position (x, y) whose successor $(x', y') = \text{next}_{d(x,y)}$ also lies outside $\mathcal{W}$, the beam propagates from (x, y) to (x', y') if and only if no agent of color c is present at (x', y'):

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W}, \\ \text{next}_d(x,y)\in P\setminus\mathcal{W}}} b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right)$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W}, \\ \text{next}_d(x,y)\in P\setminus\mathcal{W}}} \neg b_{c,d,x,y,t} \vee a_{c,x',y',t} \vee b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W}, \\ \text{next}_d(x,y)\in P\setminus\mathcal{W}}} b_{c,d,x,y,t} \vee \neg b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W}, \\ \text{next}_d(x,y)\in P\setminus\mathcal{W}}} \neg a_{c,x',y',t} \vee \neg b_{c,d,x',y',t}$$

- **Link between beam and laser variables:**
  For each laser source $(c, d, -) \in \mathcal{S}$, the laser variable $l_{c,x,y,t}$ is true at a position if and only if the beam of that source is active there:

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} b_{c,d,x,y,t} \leftrightarrow l_{c,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,-)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} (\neg b_{c,d,x,y,t} \vee l_{c,x,y,t}) \wedge (\neg l_{c,x,y,t} \vee b_{c,d,x,y,t})$$

- **Agents cannot step on active lasers:**
  An agent of color c_1 cannot occupy a position where a laser of color $c_2 \neq c_1$ is active. Agents are immune only to lasers of their own color:

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C,\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} l_{c_2,x,y,t} \rightarrow \neg a_{c_1,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C,\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \neg l_{c_2,x,y,t} \vee \neg a_{c_1,x,y,t}$$

4.3 Cooperation Detection

4.3.1 Strict Laser Semantics

Recall from that the standard encoding allows agent c to occupy a cell where a laser of color c is active — the agent is immune to its own color. The **strict** variant removes this immunity: every agent is blocked by every laser, regardless of color.

Concretely, the strict encoding adds the following constraint for every agent $c \in \mathcal{A}$, every position $(x, y) \in P$, and every time step $t \in T$:

$$\bigwedge_{c\in\mathcal{A}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \neg l_{c,x,y,t} \vee \neg a_{c,x,y,t}$$

All other constraints from remain unchanged. We call the resulting CNF formula $\Phi_{\text{strict}}(L, T)$ and the corresponding solver StrictSolver.

4.3.2 Why This Captures Cooperation

In LLE, the only cooperative act available to an agent is to position itself in the path of a laser of its own color, thereby blocking the beam and allowing other agents (who are not immune to that laser) to pass through cells the beam would otherwise reach.

An agent that blocks its own laser receives no benefit from doing so: it is immune to that laser anyway, so the blocked beam does not protect it. The sole beneficiary is a teammate.

Under strict semantics, stepping on one's own laser is forbidden. Therefore, a level that is unsolvable under strict semantics is one where every valid solution requires at least one such blocking act — i.e., genuine cooperation.

4.3.3 Formal Theorem and Proof

Theorem. Let L be an LLE level and T a time horizon. Then L requires cooperation (with horizon T) if and only if $\Phi(L, T)$ is satisfiable and $\Phi_{\text{strict}}(L, T)$ is unsatisfiable.

Proof.

($\rightarrow$) Assume L requires cooperation with horizon T. By definition, L is solvable, so $\Phi(L, T)$ is satisfiable. Suppose for contradiction that $\Phi_{\text{strict}}(L, T)$ is also satisfiable. Then there exists a valid strict trajectory σ of length T reaching all exits. In σ, no agent ever occupies a cell where its own laser is active. Since σ is also valid under standard semantics (strict constraints are a superset of standard constraints), σ is a valid standard solution in which no agent blocks its own laser. But this contradicts the assumption that L requires cooperation: every valid solution must contain a cooperative blocking act. Therefore $\Phi_{\text{strict}}(L, T)$ is unsatisfiable. $\square$

($\leftarrow$) Assume $\Phi(L, T)$ is satisfiable and $\Phi_{\text{strict}}(L, T)$ is unsatisfiable. The first condition means L is solvable. The second means there is no valid strict trajectory reaching all exits — i.e., every valid standard solution contains at least one time step t and agent c such that $a_{c,x,y,t}$ and $l_{c,x,y,t}$ are both true for some (x, y): agent c occupies a cell where its own laser is active. As argued above, this act is only beneficial to other agents. Therefore every solution requires at least one cooperative act, so L requires cooperation. $\square$

4.3.4 Practical Algorithm

The cooperation detector runs two SAT calls on the same level:

1. Run $\text{Solver}(L, T)$: if UNSAT, the level is unsolvable — stop.
2. Run $\text{StrictSolver}(L, T)$: if UNSAT, the level requires cooperation.

Both calls share the same variable factory and differ only in the additional strict constraint clauses added in step 2. The total cost is two calls to a modern CDCL solver.

Lorem ipsum dolor.

4.4 Level Generators

4.4.1 Design Pattern

All generators follow a common architecture built around three principles:

1. **SAT as oracle**: the solver is not post-hoc — it is embedded in the generation loop. A candidate level is only accepted if the solver confirms the desired property (solvability, cooperation).
2. **Separation of concerns**: level construction and property verification are decoupled. Generators build candidate levels using domain-specific heuristics; the solver decides acceptance.
3. **Extensibility**: every generator extends `BaseGenerator` and is registered via the `@register_generator` decorator, making it available to the CLI without modifying core code.

Lorem ipsum dolor.

4.4.2 Random Solvable Generator

The simplest generator. It samples a level uniformly at random (wall placement, laser sources, agent starts and exits), then queries the SAT solver. If SAT, the level is accepted; otherwise a new sample is drawn. The process repeats until a solvable level is found or a budget is exhausted.

Lorem ipsum dolor.

4.4.3 Constrained Random Solvable Generator

A structured variant that biases generation toward solvable configurations by construction, while still using the solver to verify. Constraints applied during sampling include:

- Ensuring exits are reachable from starting positions (connectivity check before SAT call).
- Limiting wall density to preserve navigable space.
- Placing laser sources only where beams have non-trivial reach.

This reduces the rejection rate compared to the fully random generator while preserving diversity.

Lorem ipsum dolor.

4.4.4 Random Cooperative Generator

Extends the random solvable generator with a second SAT call (strict solver). A level is accepted only if it is solvable (standard SAT) and requires cooperation (strict UNSAT). As established in , this guarantees that every accepted level structurally requires inter-agent coordination.

Lorem ipsum dolor.

4.4.5 Constrained Random Cooperative Generator

Combines the structural biases of the constrained solvable generator with the cooperation acceptance condition. Additionally applies heuristics that increase the probability of generating levels with cooperative structure:

- Placing laser sources such that beams cross agent paths.
- Ensuring that at least one agent's exit is behind a laser of a different color.

These heuristics increase the cooperation acceptance rate without sacrificing formal correctness: the solver remains the final arbiter.

Lorem ipsum dolor.

4.4.6 Summary

Generator	Solvable	Cooperative
Random Solvable	✓ (SAT check)	✗
Constrained Random Solvable	✓ (SAT check)	✗
Random Cooperative	✓ (SAT check)	✓ (strict UNSAT)
Constrained Random Cooperative	✓ (SAT check)	✓ (strict UNSAT)

Table 1: Overview of the four generators and their guaranteed properties.

4.5 Benchmarking Setup

Lorem ipsum dolor.

Chapter 5

Experiments

The goal of this first experimental evaluation is to compare two alternative formulations of the agent uniqueness constraint introduced in , and to measure their effect on the size of the CNF formula and on the time required by the SAT solver across levels of increasing complexity.

5.1 Experimental Setup

5.1.1 Clause Count Analysis

The two formulations differ in the number of clauses they generate to enforce that each agent occupies at most one position per time step (see for the full constraint definitions).

Global uniqueness generates a clause for every pair of distinct positions in P, for every agent and time step. The clause count per agent per time step is:

$$\binom{|P|}{2} = \frac{(|P|-1)|P|}{2} \in O(|P|^2)$$

Local uniqueness only generates clauses over pairs of positions reachable from a common predecessor in $\mathrm{next}(x, y)$, plus backward consistency. Since $|\mathrm{next}(x, y)| \leq 5$, the clause count per agent per time step is:

$$\left(\binom{5}{2} + 1\right)|P| = 11|P| \in O(|P|)$$

The theoretical crossover — where global becomes more expensive than local — occurs at $|P| = 23$: for grids with fewer than 23 walkable cells global generates fewer clauses; above this threshold local is more compact.

5.1.2 Test Levels

Four levels of increasing size and complexity were used:

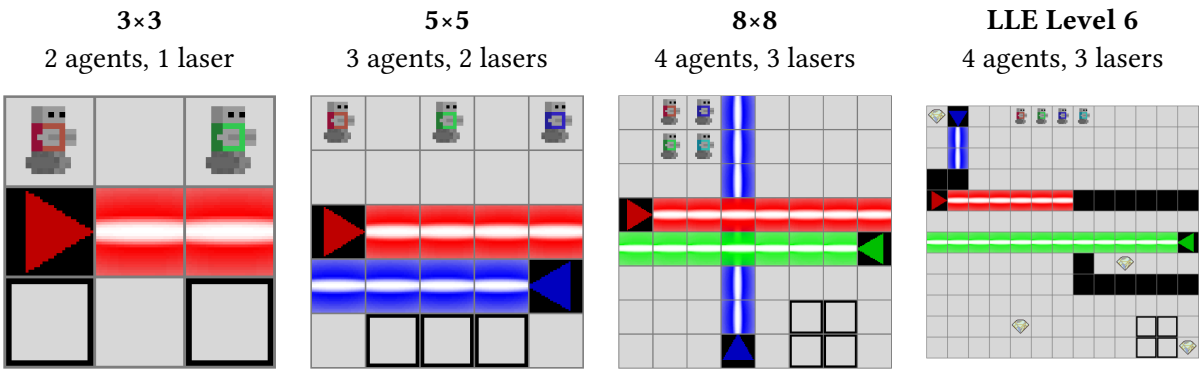

Figure 3: The four test levels used for benchmarking, ordered by increasing complexity.

The first three levels were constructed specifically to test scaling behaviour across grid size and agent count. The fourth is LLE Level 6, one of the hand-crafted levels shipped with the LLE benchmark, included to assess solver performance on a realistic cooperative level.

5.2 Results

5.2.1 Clause Count

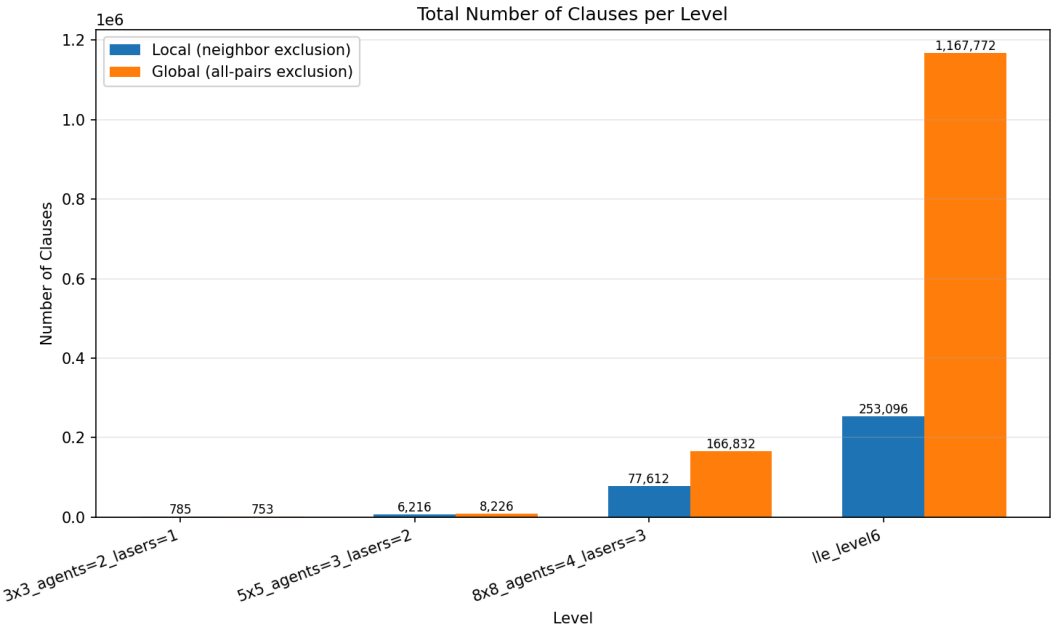

Figure 4: Number of clauses generated by the global and local uniqueness formulations across the four test levels.

5.2.2 Solving Time

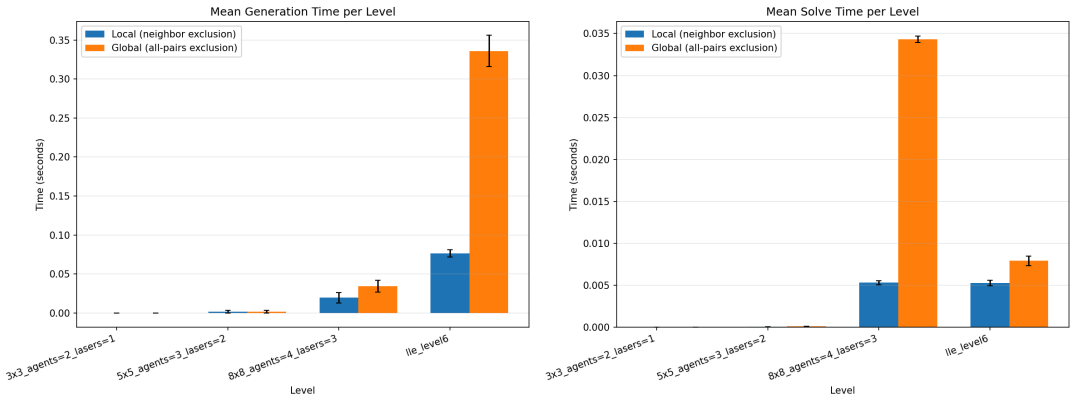

Figure 5: SAT solver time (seconds) for the global and local uniqueness formulations across the four test levels.

5.3 Discussion

Conclusion

Lorem ipsum dolor.

Bibliography

[1] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, “Laser Learning Environment: A New Environment for Coordination-Critical Multi-agent Tasks,” in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[2] Y. Molinghen, “LLE.” [Online]. Available: https://github.com/yamoling/lle

[3] S. Risi and J. Togelius, “Increasing Generality in Machine Learning through Procedural Content Generation.” [Online]. Available: https://arxiv.org/abs/1911.13071

[4] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural Content Generation via Reinforcement Learning.” [Online]. Available: https://arxiv.org/abs/2001.09212

[5] A. Summerville and M. Mateas, “Super Mario as a String: Platformer Level Generation Via LSTMs.” [Online]. Available: https://arxiv.org/abs/1603.00930

[6] G. Aloupis, E. D. Demaine, A. Guo, and G. Viglietta, “Classic Nintendo Games are (Computationally) Hard.” [Online]. Available: https://arxiv.org/abs/1203.1895

[7] M. Gumin, “Wave Function Collapse.” [Online]. Available: https://github.com/mxgmn/WaveFunctionCollapse

[8] R. Heese, “Quantum Wave Function Collapse for Procedural Content Generation,” *IEEE Computer Graphics and Applications*, vol. 44, no. 5, pp. 54–66, Sept. 2024, doi: 10.1109/mcg.2024.3447775.

[9] R. R. Torrado, A. Khalifa, M. C. Green, N. Justesen, S. Risi, and J. Togelius, “Bootstrapping Conditional GANs for Video Game Level Generation.” [Online]. Available: https://arxiv.org/abs/1910.01603

[10] A. Sarkar and S. Cooper, “Sequential Segment-based Level Generation and Blending using Variational Autoencoders.” [Online]. Available: https://arxiv.org/abs/2007.08746

[11] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey.” [Online]. Available: https://arxiv.org/abs/2003.04960

[12] J. von Neumann, *Theory of Self-Reproducing Automata*. Urbana, IL: University of Illinois Press, 1966.

[13] M. Gardner, “Mathematical Games: The fantastic combinations of John Conway's new solitaire game "life",” *Scientific American*, vol. 223, no. 4, pp. 120–123, 1970.